A substitution-stable seven-rule presentation of intuitionistic primitive recursive arithmetic

A research construction for Vladimir Reshetnikov

November 11, 2026

Abstract

The native scalar kernel in the current ConciseIPRA project has eight primitive inference schemes. We replace its two introduction schemes by one, retaining the complete original term and formula language and the ordinary meanings of both binders. An atomic matrix receives its progress hypothesis exactly when it has the syntactic form $I(y, y; u.h) = I(y, y; u.k)$; every other matrix receives its zero instance. The zero instance of every recognized matrix is provable by computation. This recovers all case introductions. A diagonal lift of an arbitrary equation then recovers all equation induction. The resulting seven-rule calculus derives exactly the original sequents and consequently inherits the whole-domain bi-interpretation with full first-order iPRA. The guard test is both preserved and reflected by capture-free substitution, so the single rule supports the usual direct structural induction on proofs.

The matched, fully typed standalone Python checker has 3,244 normalized bytes, 1,069 significant tokens and 1,124 AST nodes, compared with 3,382, 1,101 and 1,160 for the eight-rule control. This is a qualified improvement: the existing witness-selected seven-rule control is smaller at 3,205 bytes but its primitive mode is not stable under substitution. The construction does not beat the 521-byte compact profiles listed in the pinned README and does not establish absolute minimality. The new mathematics is proved in writing; the accompanying Python certificates are checked, but no new Lean or Rocq verification is claimed.

Contents

1 Result, comparison, and scope **3**

2 A substitution-stable single introduction **3**

2.1 Syntax, readings, and all seven primitive rules 4
2.2 Why the diagonal survives substitution 5
2.3 Recovering every case introduction first 6
2.4 A small logical tool with no hidden induction 7
2.5 Recovering unrestricted equation progress 7
2.6 Exact equivalence and the arithmetic consequence 8
2.7 A sufficient criterion for other guards 9
2.8 What is simpler, and what is not being claimed 10

3 Ordinary logic and the bi-interpretation with iPRA **10**

3.1 Ordinary realization and explicit logical content 10
3.2 The two term interpretations 11

3.3 Both composites and consistency 12

4 Standalone checkers and reproducible measurements 12

5 Verification and limitations 13

6 Further research questions 14

A Attribution and inherited proof material 15

B Complementary introductions with one local assumption 15
B.1 Complete native syntax and rule inventory 15
B.2 Exact equivalence and the unused-witness construction 16
B.3 Induction and numerical cases separate cleanly 17
B.4 The entire ordinary logical interface 18
B.5 Whole-domain realizations and both formula composites 19
B.6 What the theorem does and does not claim 20

C Arithmetic strength and both ordinary interpretation composites 20
C.1 The arithmetic dependency order 21

D Internal scalar predecessor bootstrap 22
D.1 Elementary terms and two finite cycles 22
D.2 A ramp with a global scaling identity 23
D.3 A comparator from scalar iteration 24
D.4 Saturated iteration gives predecessor 25
D.5 Dependency boundary 26

E From literal-equation induction to full intuitionistic PRA 26
E.1 The exact hypotheses 26
E.2 Constructor separation and two elementary tests 27
E.3 Four elementary uses of equation induction 28
E.4 An equation invariant that reflects actual equality 29
E.5 Compilation of all quantifier-free formulas 30
E.6 Recovery of induction and the full theory 31

F Pairing, primitive recursion, and internal numerical composites 31
F.1 Pairing from a bounded square-root search 31
F.2 Compiling each primitive recursion 32
F.3 Why both numerical round trips are internal 33

G The complete new checker 33

1 Result, comparison, and scope

The comparison is frozen at ProveIt commit `6bf83d8765614b0af51b35747e07fb057d6b2f42`, in `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. Its README lists thirty-eight research packages, and its Lean development proves the equivalence of the native scalar calculus, its ordinary first-order realization, and iPRA [2, 3].

The result here is a new combination of properties: seven primitive rules, the unrestricted original formula grammar, the unchanged conditional readings of the binders, and an introduction rule whose mode is invariant under capture-free numerical substitution. These are properties of the axiomatization, not just of a Python encoding.

Presentation	Rules	Binder meanings	Introduction mode under substitution
Native kernel	8	Original	Preserved; two rules
Witness-selected kernel	7	Original	Can change when a witness becomes literal zero
Uniform-progress kernel	7	Changed at equations	Preserved
Diagonal-guard kernel	7	Original	Preserved and reflected

The prior witness rule selects progress from the matrix root and whether the witness is literally zero [4]. The prior uniform-progress construction instead changes the reading of an atomic closure and modifies closure elimination [6]. The prior guarded-core construction restricts public atomic matrices beneath the paired binder to a guarded shape [5]. The present construction requires no such public restriction, changes no ordinary binder reading, and leaves closure elimination untouched.

What "simpler" means in this report. There is a strict reduction from the repository's eight-rule formalized kernel in rule count and in all three matched source-size coordinates. There is also a structural advantage over its witness-selected seven-rule alternative: substitution admissibility follows by a direct proof-tree induction without changing introduction mode. There is no claim that this candidate dominates every existing implementation, or that seven is a lower bound. In particular, the 521-byte compact profiles listed in the pinned README remain unbeaten. Those profiles assume a canonical guarded public goal and admit administrative syntax, whereas the checkers measured here validate every term, formula, scope, context and proof node in the unrestricted native language. These are different input contracts.

Target theory. Throughout, iPRA means full first-order intuitionistic arithmetic with a symbol for every fixed finite primitive-recursive definition, its defining equations, the successor separation and injectivity laws, and quantifier-free induction, with parameters and retained assumptions. The atomic-induction formulation in the usual PR language is equivalent through numerical characteristics of quantifier-free formulas. This is not merely the quantifier-free equational fragment. The conventional first-order usage is also discussed by Salehi [10].

2 A substitution-stable single introduction

The construction retains every numerical term and every formula of the native scalar calculus. It also retains the ordinary readings of its two binders. The change is confined to the introduction rule:

a particular *syntactic diagonal shape* selects progress; every other matrix selects cases. The diagonal is chosen so that its zero instance is a theorem and its recognition commutes with capture-free substitution.

2.1 Syntax, readings, and all seven primitive rules

Numerical terms and native formulas are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

In an iterator only u is bound, and only in h; in D the variables x, y are distinct. We work up to renaming of bound variables, and all substitutions avoid capture. No restriction is imposed on public formulas, intermediate formulas, iterator bodies, or witnesses. The designated contradiction remains $0 = S0$. The ordinary readings remain

$$Q_y B := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

Definition 2.1 (The diagonal predicate)**.** For a matrix $B(x, y)$, let $\mathrm{Diag}_y(B)$ mean that its *literal syntax* is

$$B(x, y) = \big(I(y, y; u.h) = I(y, y; v.k)\big) \tag{1}$$

for some bodies h, k. The displayed counter and initializer on both sides are each the variable bound as y by the impending D. The state binders u, v may be renamed independently. The bodies may use x, y, their own state variable, and any fixed parameters. This is syntactic recognition, without simplification or an equality test inside the arithmetic theory. Put

$$\eta_B(z) = \begin{cases} z, & \mathrm{Diag}_y(B), \\ 0, & \text{otherwise.} \end{cases}$$

The calculus $\mathcal{G}$ has the following complete primitive inventory. Contexts are finite reusable lists of native formulas.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \mathrm{Hyp}, \qquad \Gamma \vdash I(0, a; u.h) = a\ \ \mathrm{I0},$$

$$\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]\ \ \mathrm{IS},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)}\ \mathrm{Tr}_{=},$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \mathrm{QE},$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}\ \mathrm{DE},$$

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}\ \mathrm{DI}_{\mathcal{G}}.$$

In $\mathrm{Tr}_{=}$, $a(v), b(v)$ are numerical term templates; transport inside a compound formula is not primitive. In DE, b is fresh for Γ, A and the fixed parameters of B. In DI, z is fresh for Γ, t and

the fixed parameters of B. The displayed local assumptions are discharged. There is no primitive reflexivity, substitution, cut, ordinary connective, case, or additional induction rule.

For comparison, $\mathcal{N}$ is the eight-rule native kernel of the repository: its first six rules are the ones displayed above, and its two introductions are

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \mathrm{DI}_{\mathrm{case}}, \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}\ \mathrm{QI}_{\mathrm{eq}}, \tag{2}$$

where E is a literal equation and the same freshness conventions apply.

2.2 Why the diagonal survives substitution

Lemma 2.2 (Preservation and reflection of the guard)**.** *Let σ be a capture-free substitution for the free parameters of $B(x,y)$, with x, y retained as the distinguished bound coordinates. Then*

$$\mathrm{Diag}_y(B\sigma) \quad \Longleftrightarrow \quad \mathrm{Diag}_y(B).$$

The same assertion holds when a witness is substituted for x, provided that the witness does not contain the bound coordinate y.

Proof. Rename binders away from every term in the substitution range. A constructor-headed term remains headed by that constructor after substitution. Moreover, a term becomes the distinguished bound variable y if and only if it was already y: another retained variable stays different from y, a substituted parameter has an image not containing y, and a constructor-headed term cannot become a variable. Consequently an iterator becomes headed by the exact pair of arguments (y, y) if and only if it had that pair before substitution. The root of a formula is also preserved. An entire side that is a substituted parameter may become iterator-headed, but cannot acquire the specified shape, because its image has no free occurrence of the protected y. An existing iterator matches the shape exactly when its two displayed arguments match y, by the preceding argument. Applying these facts to both sides proves both directions of the equivalence. The bodies are unrestricted, so they impose no further condition. Witness substitution is the same argument with x among the substituted parameters. □

The reflection direction is essential. The superficially similar guard

$$I(y, 0; u.h) = I(y, 0; v.k)$$

does not have it. For a free parameter p, the matrix

$$I(y, p; u.0) = I(y, 0; v.0)$$

does not match the zero-initializer guard, whereas its substitution $p := 0$ does. Thus that test can change a case instruction into a progress instruction. Requiring the initializer itself to be the protected bound variable y prevents this event.

Lemma 2.3 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, capture-free numerical substitution, and ordinary-assumption cut are admissible in $\mathcal{G}$. Numerical substitution can be performed by induction on a derivation without changing any last-rule constructor or either mode of* $\mathrm{DI}_{\mathcal{G}}$.

Proof. Weakening, exchange and contraction change only hypothesis membership; rename eigenparameters before inserting a context that could contain them. For numerical substitution, induct on the derivation. The computation and transport rules commute with capture-free substitution. For QE and DE use the usual interchange of lifted substitution and instantiation, after renaming their bound and eigenvariables. In DI, Lemma 2.2 says that the same one of $0, z$ is retained after substitution. The substituted premise is therefore exactly the premise of the same mode of DI. This proves the induction step directly.

For cut, replace each use of a discharged ordinary hypothesis by its supplied derivation. Induct on the derivation in which that hypothesis occurs, weakening the inserted proof in each local context. Rename local eigenparameters away from the inserted proof. The guard is independent of context and proof terms, so its mode is unchanged. This proves both single-assumption cut and simultaneous replacement of hypotheses by derivations. None of these arguments appeals to the equivalence with $\mathcal{N}$ proved below. □

2.3 Recovering every case introduction first

Lemma 2.4 (Elementary equality and the diagonal base)**.** *Reflexivity, symmetry, transitivity and congruence for numerical terms are derivable using only I0 and* $\mathrm{Tr}_{=}$*. If* $\mathrm{Diag}_y(B)$*, then* $\Gamma \vdash B(t, 0)$ *for every context* Γ *and witness* t*.*

Proof. Put $r = I(0, a; u.0)$. The computation $r = a$, used as both transport premises with equation template $v = a$, gives $a = a$. From $s = t$ and $s = s$, the template $v = s$ gives $t = s$. The template $s = v$ gives transitivity, and the template $h(s) = h(v)$ gives congruence after reflexivity. At $y = 0$, both sides of (1) are of the form $I(0, 0; u.h')$, hence each equals 0 by I0. Symmetry and transitivity give their equality. No induction, body extensionality, or test for numerical equality is used. □

Lemma 2.5 (Unrestricted case introduction)**.** *Every instance of* $\mathrm{DI}_{\mathrm{case}}$ *in* (2) *is admissible in* $\mathcal{G}$*, for every native matrix* B*.*

Proof. If B does not match the diagonal, the desired rule is primitive. Otherwise start with $\Gamma, B(t, 0) \vdash B(t, Sz)$. By Lemma 2.4 the discharged base is already a theorem, so cut it out. This gives $\Gamma \vdash B(t, Sz)$. Weaken by $B(t, z)$ and apply the progress mode of $\mathrm{DI}_{\mathcal{G}}$. This recovers the omitted case instance without any general equation induction. □

Lemma 2.6 (Conditional closure and ordinary numerical cases)**.** *For every native formula template* $F(y)$*, the rules*

$$\frac{\Gamma, F(0) \vdash F(Sz)}{\Gamma \vdash Q_y F(y)}, \qquad \frac{\Gamma \vdash F(0) \qquad \Gamma, F(0) \vdash F(Sz)}{\Gamma \vdash F(t)}$$

are admissible in $\mathcal{G}$*, with* z *fresh for the retained context and fixed parameters. In particular, the second successor premise may ignore its additional hypothesis* $F(0)$*.*

Proof. Choose an unused witness coordinate x. The recovered case rule at witness 0 introduces $D_{x,y}F(y)$ from the first displayed premise. DE opens $Q_y F(y)$, which is independent of the fresh witness; return that hypothesis as the conclusion of DE. This proves the first rule. QE with the base proof proves the second rule. □

2.4 A small logical tool with no hidden induction

Only implication between two equations is needed in the completeness proof. It can be constructed directly, so no general formula selector, reflected equality, predecessor, or pairing theorem is needed at this stage. Define the numerical selector

$$C(w, a, b) := I(w, a; u.b), \qquad u \notin \mathrm{FV}(b).$$

Computation gives $C(0, a, b) = a$ and $C(Sw, a, b) = b$. For equations $U = (a = b)$ and $V = (c = d)$, with w fresh, put

$$U \Rightarrow_{=} V := Q_w(C(w, a, c) = C(w, b, d)). \tag{3}$$

This is a native formula abbreviation, not an additional connective.

Lemma 2.7 (Implication for equations)**.** *For literal equations U, V, the rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_{=} V}, \qquad \frac{\Gamma \vdash U \Rightarrow_{=} V \qquad \Gamma \vdash U}{\Gamma \vdash V}$$

are derivable using computation, equational transport, and the case-derived closure rule of Lemma 2.6.

Proof. Let $J(w)$ denote the equation inside the Q in (3). Its zero instance is interderivable with U, and each successor instance is interderivable with V, by the two selector computations and elementary equality reasoning. For introduction, convert the retained $J(0)$ to U, insert the given derivation of V, convert V to $J(Sz)$, and apply case-derived Q introduction. For elimination, convert the proof of U to $J(0)$, apply QE at $S0$, and convert $J(S0)$ to V. All conversions are between equations. This argument uses neither compound transport nor general progress. □

Lemma 2.8 (Cases in a context that mentions the inspected term)**.** *Let E be an equation and let r be any term. The rule*

$$\frac{\Gamma, r = 0 \vdash E \qquad \Gamma, r = Sv \vdash E}{\Gamma \vdash E}$$

is admissible when v is fresh for Γ, r, E. The context Γ may contain r and its free variables.

Proof. Keep Γ, r, E fixed and choose a new variable y. Set $F(y) := ((r = y) \Rightarrow_{=} E)$. Discharge the two branch equations by Lemma 2.7; this gives proofs of $F(0)$ and $F(Sv)$ in Γ. Apply the ordinary case rule of Lemma 2.6 to $F(y)$ at target r. The result is $(r = r) \Rightarrow_{=} E$. Reflexivity and implication elimination give E. The variable being generalized in the case inference was the fresh y, not a variable occurring in Γ. Thus the proof does not use the invalid convention of changing a context silently in each branch. □

2.5 Recovering unrestricted equation progress

Theorem 2.9 (The full native progress rule)**.** *For every literal equation $E(y)$, the rule*

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}$$

is admissible in $\mathcal{G}$, with the usual eigenparameter condition.

Proof. Write $E(y) = (s(y) = t(y))$ and let π be the supplied step derivation. Define the diagonal lift

$$H(y) := \big(I(y, y; u.s(y)) = I(y, y; u.t(y))\big), \qquad u \notin \mathrm{FV}(s, t). \tag{4}$$

By computation and equality alone,

$$\vdash H(0), \qquad H(Sv) \dashv\vdash E(Sv) \tag{5}$$

for every term v. The second statement uses constant state updates; it does not assert any identity between arbitrary iterator bodies.

Work temporarily in the retained context $\Delta := \Gamma, E(0)$, which contains no fresh induction variable. For fresh y set

$$\Phi(y) := H(y) \Rightarrow_{=} E(Sy).$$

There are two case proofs in this fixed context:

- At $y = 0$, substitute 0 for the eigenparameter of π. The retained $E(0)$ gives $E(S0)$. The assumption $H(0)$ may be ignored, so equation-implication introduction gives $\Delta \vdash \Phi(0)$.
- At $y = Sv$, assume $H(Sv)$. By (5) it gives $E(Sv)$. Substitute Sv for the eigenparameter of π and obtain $E(SSv)$. Discharging $H(Sv)$ gives $\Delta \vdash \Phi(Sv)$.

These uses of substitution are justified directly by Lemma 2.3; they do not invoke the result currently being proved. The ordinary case rule applies to the *compound* native formula Φ and yields $\Delta \vdash \Phi(z)$ for a fresh z. Now assume $H(z)$, use equation-implication elimination, and convert $E(Sz)$ back to $H(Sz)$. We have proved

$$\Delta, H(z) \vdash H(Sz).$$

The case inference was performed *before* adding $H(z)$ to the context, so its freshness condition is satisfied. Equivalently, one could use the explicitly justified contextual case rule of Lemma 2.8.

Take an unused witness coordinate x in the matrix $H(y)$. It has the exact diagonal shape, so primitive progress DI at witness 0 gives $\Delta \vdash D_{x,y}H(y)$. DE, with its fresh witness unused, gives $\Delta \vdash Q_y H(y)$. Together with $H(0)$, QE at a fresh successor Sz yields $H(Sz)$, hence $E(Sz)$. We have therefore constructed

$$\Gamma, E(0) \vdash E(Sz).$$

Finally apply the already recovered case-derived Q introduction to obtain $\Gamma \vdash Q_y E(y)$.

The temporary retention and final discharge of $E(0)$ are deliberate. No inference identifies conditional closure with unconditional progress. In particular, the argument also works when the base equation is false. □

2.6 Exact equivalence and the arithmetic consequence

Theorem 2.10 (Identity-language equivalence)**.** *For every finite context* Γ *and every native formula* A*,*

$$\boxed{\Gamma \vdash_{\mathcal{G}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.}$$

Both translations act effectively on finite derivations, and neither translation changes a term, formula, or retained assumption.

Proof. Translate a $\mathcal{G}$ derivation to $\mathcal{N}$ by induction. The first six rules are shared. An unguarded DI instance is native case introduction. In a guarded DI instance the matrix is an equation. The native progress rule applied to $E(y) := B(t, y)$ gives $Q_y B(t, y)$. Under the temporary base $B(t, 0)$, QE supplies $B(t, Sz)$; native case DI then introduces the required $D_{x,y}B(x, y)$.

Conversely, translate a native derivation recursively. The shared rules remain unchanged, every native case introduction is replaced by Lemma 2.5, and every native equation progress introduction is replaced by Theorem 2.9. All constructions use the original retained context and satisfy the freshness conditions. □

The proof has a strict dependency order: guard stability; structural admissibility; elementary equality; all case introductions; case-derived closure and numerical cases; implication between equations; general equation progress. It does not use equality reflection, a scalar predecessor, pairing, primitive-recursion compilation, or the eventual bi-interpretation in order to establish equivalence.

Proposition 2.11 (Transfer of the established bi-interpretation)**.** *Compose Theorem 2.10 with the native scalar calculus's established bi-interpretation with iPRA. This gives a bi-interpretation of $\mathcal{G}$ with iPRA, and equiconsistency. The numerical domains and interpreted equality are unchanged, and both numerical composites have the identity graph $x = y$.*

Proof. The first component of each composite, between $\mathcal{G}$ and $\mathcal{N}$, is literally the identity on the full syntax and on all sequents. It therefore preserves the established whole-domain interpretations, their internal formula round trips, and their numerical identity graphs. The equivalence also preserves the designated native contradiction $0 = S0$ in the empty context. Composing the finite proof translations in each direction preserves, and reflects, derivability of contradiction, which proves equiconsistency. □

2.7 A sufficient criterion for other guards

Proposition 2.12 (The guard criterion)**.** *Let $\mathsf{P}_y(B)$ be a decidable predicate on native matrices that respects capture-free renaming. Suppose that:*

1. *Every matrix satisfying P is a literal equation, and its instance $B(t, 0)$ is derivable for every witness t using only computation and equational transport.*
2. *Every equation $E(y)$ has an equation lift $H(y)$, independent of an added unused witness coordinate x, such that $\mathsf{P}_y(H)$ holds and $H(Sv) \dashv\vdash E(Sv)$ is derivable using only computation and equational transport, for arbitrary v.*

Replace the diagonal test in the seven-rule presentation by P. The resulting calculus $\mathcal{G}_\mathsf{P}$ derives exactly the native sequents. If, in addition, P is preserved and reflected by capture-free numerical substitution, its structural substitution theorem follows by direct rule-preserving induction.

Proof. The first condition recovers every missing case introduction exactly as in Lemma 2.5. Hence the case-derived closure, numerical cases and equation-implication rules are available. The second condition supplies the only properties of the lift used in Theorem 2.9. Since the base and endpoint conversions belong to the fragment with computation and equational transport, neither condition presupposes the general progress rule being recovered.

For clarity, equivalence does not silently assume numerical substitution in $\mathcal{G}_\mathsf{P}$ when the optional stability condition is absent. Translate native derivations by strong induction on their height. In the native progress case, substitute 0 and Sv into its native premise derivation first. Native

substitution preserves derivation height, so both resulting proofs still have smaller height than the progress conclusion and can be translated by the induction hypothesis. They supply the two Φ branches in the displayed proof above. Renaming, weakening and cut require only the stated renaming invariance. The other native rules translate as before, and the reverse translation uses only the fact that a progress-selected matrix is an equation. This proves the sequent equivalence. Under the additional preservation and reflection condition, numerical substitution instead follows directly from the proof of Lemma 2.3, with no change of mode. □

2.8 What is simpler, and what is not being claimed

Relative to the eight-rule native kernel, this construction removes one primitive introduction while retaining its full language, all witnesses, and both original binder readings. Relative to the earlier seven-rule witness-selected presentation, its improvement is structural: the mode depends only on protected bound-variable positions in the matrix, so ordinary substitution is proved by a direct rule-preserving induction. There is no exceptional substitution-repair proof.

The earlier guarded-language proposal restricted the public atomic matrices; here every original formula remains available and the comparison theorem is the identity on *all* sequents. The earlier uniform-progress proposal changes the meaning of Q at equations and modifies its elimination; here neither change is made. The diagonal initializer supplies the missing combination of a provable zero base and a substitution-invariant test.

This is not a proof of global minimality. A seven-rule count is not a new record by itself, and the syntactic guard has an implementation cost. Any claim about a smaller complete checker must be supported by the separate matched source measurements, including syntax validation, substitution, freshness, and the public certificate interface. The new equivalence above is a written mathematical proof; it is not, by itself, a claim that a Lean or Rocq verification of this new kernel has been compiled.

3 Ordinary logic and the bi-interpretation with iPRA

The equality of complete consequence relations established above is what allows the existing arithmetic bridge to be reused. Agreement on closed numerals or on equational theorems alone would not suffice.

3.1 Ordinary realization and explicit logical content

Let T_I be intuitionistic first-order logic with actual equality, the scalar terms of the new calculus, its two iterator computation schemes, and the following additional arithmetic principles:

$$\forall x\,(x = 0 \vee \exists z\, x = Sz), \qquad (0 = S0) \to \bot,$$

together with contextual induction for literal equations. These are principles of the ordinary realization; in the native seven-rule calculus they are derived, as are the usual logical inference rules. The realization is

$$(s = t)^\circ = (s = t), \quad (Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$

$$(D_{x,y} B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

All interpretation domains are the whole numerical domain, defined by $x = x$. No quotient equality is used.

For clarity, there are no unstated primitive logical rules in the new calculus. Their native definitions may be written using the selector $C(v,a,b) = I(v,a;u.b)$, a formula selector $M_v(A,B)$, and a formula gate G_vA. At equations the selector applies C to corresponding sides and the gate has zero on both sides at its zero branch. At binders these operations recurse through the formula; unmatched binder depths are padded using an equivalent dummy-witness gated formula. First replace any primitive Q_yB by the interderivable $D_{x,y}B$ with an unused witness coordinate x; the case-derived closure rules prove this in both directions. The explicit recursive definitions on the resulting Eq/D syntax and their endpoint proofs are reproduced in Appendix B.

With $p(v) = I(v,0;u.C(u,1,0))$, one may use the following full set of macros (fresh bound variables are always chosen):

$$\begin{aligned}
\bot_* &= (0 = S0),\\
A \Rightarrow_* B &= D_{u,z}M_z(A,B),\\
A \wedge_* B &= D_{u,z}G_zM_{p(z)}(A,B),\\
\exists_* x\, A(x) &= D_{x,y}G_yA(x),\\
A \vee_* B &= \exists_* x\, M_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* D_{u,z}G_zA(z).
\end{aligned}$$

The variable u is unused. The case rules give all ordinary intuitionistic introduction and elimination rules for these macros. The progress rule supplies exactly the additional equation induction. No excluded-middle rule for arbitrary formulas is added.

The required separation sentence in T_I must not be omitted: positive equations alone admit a singleton model. In the native reading, $0 = S0$ is the designated contradiction and explosion from it is derivable by the selector equations. The ordinary separation sentence is precisely what connects that designated equation with ordinary first-order falsity.

3.2 The two term interpretations

An iterator in iPRA is defined by primitive recursion with an update that ignores the recursion counter:

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

This is an external translation on each fixed finite term description; there is no internal universal evaluator for all primitive-recursive programs.

In the other direction the scalar theory first constructs predecessor, then a faithful equality characteristic, then quantifier-free induction, and then pairing and projections. The dependency order matters: pairing and equality reflection cannot be used as assumptions in the predecessor construction. Complete inherited arguments are reproduced in Appendices D, E and F.

If π, L, H are the resulting pairing and projections, a fixed PR definition

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n,f(n,\vec{p}),\vec{p})$$

is compiled as

$$\begin{aligned}
V(n,\vec{p}) &= I(n,\pi(0,g^*(\vec{p}));u.\pi(SL(u),h^*(L(u),H(u),\vec{p}))),\\
f^*(n,\vec{p}) &= H(V(n,\vec{p})).
\end{aligned}$$

Equation induction proves $L(V(n,\vec{p})) = n$, and the projection identities prove the defining equations of f^*.

3.3 Both composites and consistency

For each fixed iterator and each fixed PR definition, equation induction proves equality of two fixed term realizations satisfying the same defining equations. External induction on finite descriptions therefore identifies every operation with its double translation. The numerical composite maps are the identity and are internally represented by $x = y$.

For formulas, the original native-to-ordinary and ordinary-to-native translations already have both provable round trips. The substantive paired-binder round trip is

$$D_{x,y}B \quad \dashv\vdash \quad \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big).$$

The closure round trip is obtained using a dummy witness. Because the new calculus has exactly the original sequents, every such round-trip proof is also a proof in the new calculus. Thus the composition with the T_I–iPRA bridge is a genuine bi-interpretation on the whole numerical domain, not merely a mutual consistency reduction.

Effective transformations of finite derivations carry a proof of native $0 = S0$ to an iPRA proof of $\bot$, and conversely. This yields equiconsistency. It asserts neither that iPRA proves its own consistency nor that the new system has a special standard-domain semantics.

4 Standalone checkers and reproducible measurements

The three compared checkers share the same explicit de Bruijn syntax, ordinary JSON input, scope checks, equality rule and input/output contract. They differ only in their introduction rules. The eight-rule control is the repository's `one_context/direct/native_checker.py` at the pinned commit; the witness and diagonal variants are matched edits of that control. The filename of the control records its package location, not a change to the two-template equality rule used in this comparison.

The data object has exactly four fields:

```
{"variables": n, "context": [...], "goal": formula, "proof": proof_tree}
```

Here n is a nonnegative integer. Context entries and the goal must be well-scoped formulas. Terms are tagged as `v`, `z`, `s`, and `i`; formulas as `=`, `q` and `d`. Only an iterator's body binds its state variable. The matrix of D binds y at index 0 and x at index 1. All proof nodes are checked and synthesize their conclusions.

The new introduction is implemented by opening the witness and retaining the opened equation itself exactly when both iterator prefixes are `['i', ['v', 0], ['v', 0]]`; otherwise it retains the shifted zero instance. Well-formedness is checked before testing these prefixes. Witness substitution cannot manufacture the bound y in one of these positions, so checking the guard after opening the witness is equivalent to checking the original matrix.

Acceptance means exit code zero and exactly `ACCEPT` followed by a newline. The programs use neither assertions for logical checks nor `eval`, a proof database, an external prover, a trusted compiler, or an imported axiom file. The untrusted proof compiler and test producer are separate from the checker. The trusted runtime includes CPython and its standard `json` and `sys` modules, just as the controls do. Resource exhaustion may reject a large valid finite certificate; no claim of fixed-machine resource completeness is made.

Rule correspondence. The implementation uses one branch for each displayed primitive rule. By induction on a successful call to `infer`, its result is a well-scoped formula derivable from the

supplied context: the hypothesis branch checks membership, the computation branches check their term arguments, and transport and QE compare the exact substituted minor premise. DE checks its continuation against a shifted conclusion under a new variable, excluding eigenvariable escape. DI checks exactly the single local hypothesis selected by the guard. The binder increments in `walk` are zero for ordinary term arguments, one for an iterator body and a Q matrix, and two for a D matrix; structural induction gives the capture-free meanings of `shift` and `sub`. Conversely every finite derivation of the displayed calculus has the corresponding tagged certificate. This is a written correspondence argument for the idealized unbounded-resource program, not a proof-assistant-checked theorem about the Python implementation.

Complete checker	Bytes	Tokens	AST nodes	gzip-9
Native, eight rules	3,382	1,101	1,160	990
Witness-selected, seven rules	3,205	1,041	1,099	975
Diagonal-guard, seven rules	3,244	1,069	1,124	989

The normalization applies `ast.unparse(ast.parse(source))` and appends one line-feed byte, under CPython 3.12.14. Significant tokens exclude encoding, indentation, dedentation, newlines, comments and the end marker. AST nodes are counted with `ast.walk`. Informational gzip sizes use compression level 9 and `mtime=0`. The same procedure is used for all three modules; none of these figures omits a checker helper.

The diagonal rule saves 138 normalized bytes (4.08%), 32 tokens (2.91%) and 36 AST nodes (3.10%) against the eight-rule control. It costs 39 bytes against the witness-selected control. This is a Pareto tradeoff between source size and a substitution-invariant primitive interface, not a new record for unrestricted minimum source size.

5 Verification and limitations

The reproducible implementation checks used 15 named proof fixtures and 17 additional malformed or scope-attack fixtures. Every fixture ran against all three standalone checkers, both normally and under `python -O`: 192 processes in total, with zero mismatches. The positive fixtures include a diagonal-guard induction whose hypothesis is used, the derived identity closure $Q_y(I(y, 0; u.Su) = y)$, nonvacuous existential elimination, compound Q and D matrices, and a nested capture-avoiding witness. Expected rejections distinguish ordinary atomic cases from progress, reject a premature eigenvariable, and reject unrestricted compound induction.

There were also 240 finite checks of guard preservation and reflection under parameter substitution or witness opening, together with an explicit counterexample to stability of the older zero-seed guard. Independent standard-natural-number evaluation confirmed 80 capture-free term substitutions, with no mismatches or budget skips. These tests check the implementation and boundary conditions; they do not replace the written metatheory.

Executable translations. The untrusted translator implements both complete proof-tree translations, including elimination of arbitrary native `qi` nodes. A separate suite checks 29 certificate cases, with 93 checker validations covering every source rule in both calculi and both round trips. In particular it compiles an identity induction, an induction whose base is the unproved equation $a = b$, and an induction using both the current induction hypothesis and a retained parameter equation. The three translated proofs are also accepted through the public command-line interface in ordinary and optimized Python modes. The count separates 87 direct calls to the standalone checker modules from six command-line processes.

Certificate	Original JSON bytes	Translated JSON bytes
Identity induction	485	11,506
Conditional seed equality	885	14,218
Contextual update congruence	1,965	20,283

These are certificate sizes, not checker sizes. The larger proof trees make explicit the logical work removed from the primitive kernel. The report claims an effective translation and reports these examples; it does not claim a polynomial bound on serialized tree size.

The checker tests establish that the supplied finite certificates are processed as reported. They do not establish universal soundness, completeness, or consistency. Those claims rest on the written metatheorems and the inherited arithmetic bridge. The pinned repository's Lean formalization verifies the original bridge; it does not automatically verify a new Python checker or the new diagonal-rule comparison. No new Lean or Rocq build was performed for this delivery.

The archive includes the new construction, checker sources, proof translation code and examples, measurements, reproduction scripts, provenance, and the inherited arithmetic arguments with attribution. The report's new contribution is the diagonal dispatch theorem and its matched implementation. It does not claim a new predecessor construction, a new PR compilation method, an absolute minimum number of rules, or a smaller version of the 521-byte guarded-profile checker listed in the pinned README.

6 Further research questions

1. **Lean comparison with the existing native kernel.** Add a deeply embedded $\mathcal{G}$ with the guard as a structural predicate. Prove guard preservation and reflection first, followed by substitution and cut directly. Then implement the equation-only implication construction and the proof of unrestricted progress. The target is an exact sequent theorem, not only consistency.

2. **The smallest stable guard.** Can a cheaper decidable structural class retain all three required properties: provable zero instances, a lift of arbitrary equations, and reflection under substitution? The diagonal guard is a concrete solution, not a lower-bound theorem.

3. **Proof size and sharing.** The translations are effective, but syntactic lifting and repeated local derivations may expand tree certificates. Determine explicit size bounds and compare trees with a checked DAG format. Sharing must not enlarge the trusted logical rule set implicitly.

4. **Combining independent equality reductions.** The repository's crossed one-template transport is compatible at the mathematical level with the present introduction theorem. A matched implementation could determine whether their combination improves the smallest fully typed checker while retaining a transparent proof format. Such a measurement is separate from the results reported here.

5. **A fair compact-profile comparison.** Can the new substitution-stable rule be encoded within a compact administrative representation with a complete public-goal validator, and improve its normalized source size? The current typed experiment does not answer that question.

A Attribution and inherited proof material

The following five sections reproduce the mathematical bridge from the pinned repository's `Research/certified_zero/reference/` material with light introductory wording changes. The Choice-DI theorem and its logical macros, the ordinary realization, scalar predecessor bootstrap, equality reflection, quantifier-free induction, pairing and PR compilation are inherited results, not new results of this report. Exact fetched sources and their provenance are included separately in the archive. The repository distributes these materials under the MIT No Attribution license; its license text is included.

B Complementary introductions with one local assumption

This inherited comparison starts from a paired introduction retaining both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The inherited Choice-DI interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{6}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{7}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{8}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{9}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{10}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (10) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{11}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (10); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{12}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (11), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{13}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{14}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (14). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (15) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (16) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (17)$$

The base is (16). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (18)$$

The first uses the comparator recurrence. The second uses (17) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (19)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (20)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (19) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{21}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{22}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{23}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{24}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{25}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (25) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{26}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{27}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{28}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (21). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (27); symmetry and (24) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{29}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{30}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{31}$$

To prove the range assertion, split (23). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (22) also give, for all u, v,

$$C(u,v,v) = v. \tag{32}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{33}$$

If $u = 1 = S0$, (28) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{34}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{35}$$

Using these equations and (25), we now derive

$$0 + y = y, \tag{36}$$
$$(Sx) + y = S(x + y), \tag{37}$$
$$d(Sx, Sy) = d(x, y), \tag{38}$$
$$d(Sa, a) = 1. \tag{39}$$

Each induction below has a single equation as its induction formula.

For (36), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (37), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (37) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{40}$$

Indeed, both sides are $S(y + r)$.

For (38), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (39). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (38) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{41}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (36) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (32).

For the step, assume (41) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (41) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (40) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (41). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{42}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (41). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (39) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{43}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{44}$$

$$e(a,b) = 0 \ \vee \ e(a,b) = 1. \tag{45}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (30), which is equivalent to $d(Sa,b) = 1$ by (33), and then to $a = b$ by (42). The range assertion is (29) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (44) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{46}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{47}$$

$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{48}$$

$$b_{A\vee D} = C(b_A, 0, b_D), \tag{49}$$

$$b_{A\to D} = C(b_A, b_D, 0). \tag{50}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{51}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (44)–(45). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (51) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (51), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G The complete new checker

This is the full standalone program. Its source-size coordinates include all the code below. The shorter normalized rendering differs only in formatting.

```
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def term(a, n):
    match a:
        case ['v', i]:
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')


def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')
```

```
def walk(a, f, d=0):
    if a[0] == 'v':
        return f(a[1], d)
    return [a[0]] + [walk(b, f, d + ({'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a)))[j])
                     for j, b in enumerate(a[1:])]


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])


def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop])


def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['zero', a, h]:
            term(a, n)
            term(h, n + 1)
            return ['=', ['i', ['z'], a, h], a]
        case ['succ', c, a, h]:
            term(c, n)
            term(a, n)
            term(h, n + 1)
            return ['=', ['i', ['s', c], a, h], sub(h, ['i', c, a, h])]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = a if a[0] == '=' and a[1][:3] == a[2][:3] == ['i', ['v', 0], ['v', 0]] else shift(sub(a, ['
    z']))
            require(infer(p, [h] + [shift(x) for x in g], n + 1) == sub(a, ['s', ['v', 0]], 0))
            return ['d', b]
        case ['tr', a, b, p, q]:
            term(a, n + 1)
            term(b, n + 1)
            e = infer(p, g, n)
            require(e[0] == '=')
            s, t = e[1:]
            require(infer(q, g, n) == ['=', sub(a, s), sub(b, s)])
            return ['=', sub(a, t), sub(b, t)]
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
```

```
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `6bf83d8765614b0af51b35747e07fb057d6b2f42`. https://github.com/VladimirReshetnikov/ProveIt/tree/6bf83d8765614b0af51b35747e07fb057d6b2f42/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt contributors, *Native.lean* and *NativeStruct.lean*, same commit, `Lean/ConciseIPRA/`.

[3] ProveIt contributors, *NativeBiInterpretation.lean* and *BiInterpretation.lean*, same commit, `Lean/ConciseIPRA/`.

[4] ProveIt contributors, *witness_rule* research package and *NativeOneIntro.lean*, same commit.

[5] ProveIt contributors, *guarded_core* research package, same commit. See also *guarded_source* and *deterministic_auxiliary*.

[6] ProveIt contributors, *uniform_progress* research package, same commit.

[7] ProveIt contributors, inherited *scalar_predecessor.tex*, `Research/certified_zero/reference/`, same commit.

[8] ProveIt contributors, inherited *equality_reflection.tex*, `Research/certified_zero/reference/`, same commit.

[9] ProveIt contributors, inherited *primitive_recursion.tex*, `Research/certified_zero/reference/`, same commit.

[10] S. Salehi, *Decidable formulas of intuitionistic primitive recursive arithmetic*, Reports on Mathematical Logic 36 (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.